



Day 5: Building Dynamic Web Pages with JavaScript & jQuery

FULL STACK BOOTCAMP · DAY 5 OF 13

DURATION: 2 HRS 30 MIN

From Static Pages to Data-Driven Applications

How Do Netflix, Amazon & LinkedIn Generate Thousands of Cards?

Open Netflix. Open Amazon. Open LinkedIn. Every one of them shows hundreds – sometimes thousands – of cards on screen, each with a photo, a name, a rating, a title.

- ❓ Did a developer hand-write 500 separate HTML cards? Or is something smarter going on?

Pause. Think about it. Today, **you'll** find out exactly how it's done – and build it yourself.





Today's Mission: Student Directory

Every concept you learn today feeds directly into one real application — a fully dynamic Student Directory. By the end of this session, you'll have built it from scratch.

1

Search Box

Filter students by name in real time

2

Student Cards

Dynamically generated from data

3

Detail View

Click a card to expand full profile

4

Show / Hide

Toggle info with smooth animations

❏ Every module today is a building block. Nothing is filler — everything ships in the final product.

Rapid Fire Quiz — Pair Up!

Before we build forward, let's confirm your foundations are solid. Work with your partner — instructor will call on pairs at random.

1 **let vs const**
What is the key difference between let and const?

2 **getElementById**
How do you select an element using getElementById()?

3 **onclick vs addEventListener**
What is the practical difference between the two event approaches?

4 **Validation**
Write the condition to check if a textbox is empty.

5 **DOM**
In your own words — what is the DOM and why does it matter?

Why Do Arrays Exist? + Arrays in Action

The Problem

Imagine storing 500 student names. Would you really write this?

```
let student1 = "Rahul";  
let student2 = "Ankit";  
let student3 = "Priya";  
// ... 497 more lines
```

The Solution: Arrays

An array stores multiple values in a single variable — ordered, indexed, and infinitely scalable.

```
const students = ["Rahul", "Ankit", "Priya", "Sara",  
"Dev"];  
  
console.log(students);           // Full array  
console.log(students[0]);        // "Rahul" — index starts  
at 0  
console.log(students[4]);        // "Dev" — last item  
console.log(students.length);    // 5 — total count
```

Unmanageable. Unmaintainable. Unprofessional.

Index

Arrays are zero-indexed — first item is `[0]`, not `[1]`

.length

Returns the total number of items in the array

Last Item

Always at index `array.length - 1`



Arrays: Mission & Code Challenge

🏆 Code Challenge — No Googling/ChatGPT/AI

1

Favourite Movies

At least 5 items. Print the first and last item to the console.

Print Middle Item

Use `.length` and `Math.floor()` to always find the exact middle.

2

Favourite Games

At least 5 items. Print the total length of your array.

Print Last Item

Use `length - 1` as the index — never hardcode the last position.

3

Favourite Foods

At least 5 items. Access the middle item by index and print it.

Total Items

Print: "You have 5 movies in your list." using a template literal.

🌟 Bonus

Use `prompt()` to ask the user which movie they like. Print it back.

❏ No copy-paste! Raise your hand if stuck.

Why Do Objects Exist? + Your First Object

An Array Has Limits

An array stores a list of values — but every value is anonymous. How do you know which item is the name? Which is the CGPA?

```
// Confusing — what is what?  
const student = [  
  "Rahul", "CSE",  
  8.4, "Jaipur"  
];
```

Enter: JavaScript Objects

Objects store data as key-value pairs — like a student ID card. Every field has a name (key) and a value.

```
const student = {  
  name: "Rahul",  
  branch: "CSE",  
  cgpa: 8.4,  
  city: "Jaipur"  
};  
  
console.log(student.name);    // "Rahul"  
console.log(student.branch);  // "CSE"  
console.log(`${student.name} studies ${student.branch}  
with CGPA ${student.cgpa}`);
```

Key

The property name — always a string identifier (no quotes needed)

Value

Any data type — string, number, boolean, even another array or object

Dot Notation

object.key — the cleanest, most common way to read a property



Objects: Mission & Code Challenge

Mission — 8 Minutes • Solo

Create a JavaScript object called `myProfile` with at least 7 properties. Then print a natural-sounding sentence to the console using template literals.

1

Required Fields

name, branch, year, cgpa, city, skills (array), hobbies (array)

2

Output Target

```
I am Deepak from CSE, Year 2, CGPA  
8.7. Currently in Jaipur.
```

3

Stretch Goal

Store skills as an array inside your object. Print `myProfile.skills[0]` – your top skill.

Why Do Loops Exist? + Loops in Action

The Problem

Without loops, generating 500 student cards means writing 500 lines of repetitive code.

1

1 Student Card

One card, written manually

2

10 Student Cards

Getting tedious fast

3

Hundreds of Cards

Completely impossible by hand



Would you manually
create every card?

The Solution: for Loop

A loop runs the same block of code for every item in an array – automatically.

```
const students = ["Rahul", "Ankit", "Priya", "Sara", "Dev"];

for (let i = 0; i < students.length; i++) {
  console.log(students[i]);
}

// Output:
// Rahul
// Ankit
// Priya
// Sara
// Dev
```

1

Array

Your data source

2

Loop

Iterates every item

3

HTML Cards

Generated automatically

4

Browser

Renders the result

Loops: Mission & Code Challenge

Mission — 10 Minutes · Solo

Using the array and loop, generate Bootstrap student cards dynamically in the browser.

01

Create Array

Array of 5 student objects with name, branch, cgpa

02

Write the Loop

for loop that iterates through every student

03

Build HTML String

Construct a Bootstrap card inside the loop

04

Inject to DOM

Use innerHTML to render all cards in the browser



Serial Number

Display $i+1$ as a badge on each card



Alternate Colors

Use $i \% 2 === 0$ to alternate card background



CGPA Badge

Show a different badge color for $CGPA > 8$



Bonus

Display total number of students at the top of the page

Arrays + Loops = Web Pages

This is the WOW moment. Watch how data becomes a webpage in real time.

Before: Static HTML

Every card is hardcoded. Change the data? Rewrite the HTML.

```
<div class="card">
  <h5>Rahul</h5>
  <p>CSE | CGPA: 8.4</p>
</div>
<div class="card">
  <h5>Ankit</h5>
  <p>ECE | CGPA: 7.9</p>
</div>
<!-- 498 more cards... -->
```

After: Dynamic HTML

One loop. Any number of students. Update the array — the page updates itself.

```
const students = [
  { name: "Rahul", branch: "CSE", cgpa: 8.4 },
  { name: "Ankit", branch: "ECE", cgpa: 7.9 },
  { name: "Priya", branch: "IT", cgpa: 9.1 }
];

let html = "";
for (let i = 0; i < students.length; i++) {
  html += `
    <div class="card p-3 mb-3">
      <h5>${students[i].name}</h5>
      <p>${students[i].branch} | CGPA: ${students[i].cgpa}</p>
    </div>`;
}
document.getElementById("container").innerHTML = html;
```

innerHTML

Injects HTML string directly into the DOM

Template Literals

Backticks allow embedding variables inside HTML strings

append()

jQuery alternative — adds content without replacing existing content

📌 This is how every modern web app works. Arrays → Loop → innerHTML → Browser.

Why jQuery? + jQuery in Action

Vanilla JavaScript

Verbose. More typing. More room for error.

```
document.getElementById("title")
element.addEventListener("click", fn)
element.textContent = "Hello"
element.style.display = "none"
element.style.display = "block"
```

jQuery — Same Result, Less Code

One line. Readable. Chainable. Industry standard for DOM interaction.

```
$("#title")
$("#title").click(fn)
$("#title").text("Hello")
$("#title").hide()
$("#title").show()
```

```
// Toggle student details with one line
$("#btn").click(function() {
  $(".details").slideToggle();
});
```

```
// Also useful:
$(".card").fadeIn();
$(".card").fadeOut();
$(".card").hide();
$(".card").show();
```

Selectors

`$("#id"), $(".class")` — same as CSS selectors

click()

Attach a click event handler

text() / html()

Get or set content

show() / hide() / toggle()

Control visibility instantly

❑ jQuery doesn't replace JavaScript — it simplifies it. You still need to understand JS first.

jQuery: Mission & Code Challenge

Every student card should have a 'Show Details' button. Clicking it expands hidden details. Clicking again collapses them.

1

Click Button

User clicks "Show Details"

2

Expand Details

Hidden section slides open with `slideToggle()`

3

Click Again

User clicks the same button

4

Collapse Details

Section slides closed smoothly

```
$(".btn-  
details").click(function() {
```

```
$(this).closest(".card").find(".  
details").slideToggle();  
});
```

🔥 Code Challenge — Level Up



Change Button Text

When details expand, button text changes to "Hide Details". When collapsed, back to "Show Details".



Animate Color

Button background changes color on toggle using jQuery `.css()`



🌟 Bonus

Add a `fadeIn()` animation when the page first loads

Spot the Bug — Broken Student Directory

Find At Least 5 Errors

```
const students = [
  { name: "Rahul", branch: "CSE", cgpa: 8.4 },
  { name: "Ankit", branch: "ECE", cgpa: 7.9 },
  { Name: "Priya", branch: "IT", cgpa: 9.1 } ];

let html = "";
for (let i = 0; i <= students.length; i++) {
  html += `<div class="card">
    <h5>${students[i].name}</h5>
  </div>`;
}
document.getElementById("container").innerHTML = html;

$(".btn").click(function() {
  $(".details").slideToggle;
});

$("#search").keyup(function() {
  let val = $(this).val();
  $(".card").each(function() {
    let name = $(this).find("h5").text();
    if (name.includes(val)) {
      $(this).show();
    } else {
      $(this).hide();
    }
  });
});
```